Sevilla(España), 26 de Octubre de 2026

Al Profesor de la Asignatura
Proyecto de Ingeniería de Software
Grado en Ingeniería Informática
Universidad Internacional de Valencia
Dr. Horacio Kuna
S/D

De mi mayor consideración:

Me dirijo a Ud. a efectos de presentar la propuesta para el desarrollo de un producto software como tema para la cátedra “Proyecto de Ingeniería de Software” correspondiente al Grado en Ingeniería Informática de la Universidad Internacional de Valencia.

El producto software a desarrollar se denomina: Comparador de precios de supermercado.

Sin otro particular, y quedando a la espera de la evaluación de la propuesta, me despido atte.



Daniel Villarreal Bocanegra

Contenidos

Planteo del problema **2**

Introducción y objetivos **2**

Alcance y limitaciones **2**

Especificación de módulos **2**

Funcionales 2

No funcionales 2

Descripción de los módulos **2**

Procesos automatizados **3**

Estimación de tamaño por módulo **3**

Selección de módulos a implementar **4**

Módulos a programar (debe ser alrededor del 20% del proyecto): **4**

Entorno tecnológico y metodológico **4**

Planificación de actividades **5**

Planteo del problema

El problema a resolver es poder comparar fácilmente entre los precios de los productos de supermercado de los diferentes comercios cercanos, facilitando también la gestión y optimización de la lista de la compra cotidiana.

Introducción y objetivos

El proyecto surge para dar respuesta a la creciente fragmentación de precios en los supermercados locales y a la dificultad diaria que enfrentan los consumidores para optimizar su presupuesto en la cesta de la compra. Ante la variabilidad de tarifas entre distintas cadenas de distribución, los usuarios carecen de una herramienta unificada y dinámica y que les permita conocer en tiempo real dónde adquirir los productos al menor coste global. La principal limitación para comparar estos precios reside en la obtención de datos, ya que muchos supermercados dificultan la visibilidad online de los precios de sus productos lo que nos hace descartar el scrapping como método principal. Esto nos lleva a elegir el sistema de crowdsourcing, donde serán los propios usuarios los que hagan la labor de recopilar los precios. Para facilitar la introducción de datos se les proporcionará la facilidad de subir directamente el ticket de compra del supermercado.

Objetivos principales:

- **Optimizar la experiencia de compra del usuario** mediante la creación inteligente de listas de la compra, cálculo del coste estimado en diferentes comercios y trazado de rutas óptimas con integración de Google Maps.
- **Implementar sistema de crowdsourcing** impulsado por la lectura automatizada de tickets de compra mediante modelos de lenguaje (LLM) y un sistema de reputación y gamificación para garantizar la fiabilidad de los datos aportados por la comunidad.
- **Ofrecer analíticas de mercado** mostrando la evolución histórica y la variación de precios de los productos a lo largo del tiempo y la mejora en la economía doméstica.

Alcance y limitaciones

Alcance:

- Registro, autenticación y gestión de perfiles de usuario con sistema de reputación y gamificación.
- Catálogo de productos estructurado por categorías, marcas, plataformas de venta, precios y sucursales geolocalizadas.
- Módulo de subida y procesamiento de tickets de compra de forma manual y alternativamente mediante IA (LLM) para la extracción automática de precios y actualización colaborativa de la base de datos.
- Algoritmo de análisis de tickets para evaluar validez y legitimidad de los datos.

- Sistema de reputación y beneficios basado en aportaciones.
- Creación, edición y gestión de listas de la compra inteligentes. Con encuesta trás compra para validación de precios.
- Algoritmo de cálculo de costes y optimización de ruta de visita a los supermercados, integrado con la API de Google Maps.
- Visualización de gráficos estadísticos de variación de precios por producto y comercio.
- Sistema de gestión de roles y permisos.
- Backend para administradores donde gestionar todas las entidades y auditar las interacciones y datos introducidos.

Limitaciones:

- No se incluirán pasarelas de pago ni la posibilidad de realizar compras directas o comercio electrónico dentro de la aplicación; el enfoque es estrictamente de comparación, planificación y optimización.
- La actualización masiva de precios dependerá exclusivamente de la participación de la comunidad (crowdsourcing) y la validación por LLM, sin integraciones directas o automáticas con APIs oficiales de grandes superficies. Aunque sí que habrá un panel de administración como medida de seguridad donde poder gestionar todos los recursos. En una posible futura versión se plantean posibles asociaciones con comercios para la facilitación de listados de precios o beneficios para los usuarios, pero no para el MVP.
- Para evitar violaciones de LOPD el modelo de LLM no registrará datos personales tales como nombre, DNI o números de cuenta que pudiesen aparecer en los tickets. Igualmente, se informará a los usuarios de cuales son los posibles datos sensibles y se les animará a ocultarlos.
- El sistema de crowdsourcing requiere de muchos usuarios colaborando simultáneamente, por lo que existe riesgo de precios desactualizados en fase de implantación de la aplicación. La aplicación se centrará inicialmente en un ámbito geográfico provincial (Sevilla) para simplificar la validación del catálogo de sucursales y reducir los efectos comentados.

Especificación de módulos

Funcionales

- Autenticación mediante correo y contraseña o a través de cuenta de Google.
- Gestión de comercios cercanos. Incluye dar de alta comercios para posterior validación y marcar comercios como cercanos y favoritos. Permite también establecer áreas con comercios de interés.

- Gestión de listas de la compra. Se integra con el buscador de productos y comparador de precios. Permite gestionar listas de la compra pasadas, marcarlas como favoritas y compararlas con los precios actuales.
- Buscador y comparador de precios. Permite buscar productos por nombre, categoría, precio y cercanía. Cada precio va indicado con un porcentaje de seguridad que indica cuántas personas han confirmado el precio y el tiempo pasado desde la última confirmación.
- Subida de producto. El usuario puede subir un producto con su precio manualmente indicando el comercio. Alternativamente puede subir un ticket de compra donde con LLM se registran los datos de productos y precios. En caso de que haya datos que no se comprendan se pedirá al usuario que rellene un formulario en el que solo deberá indicar esos datos faltantes. En caso de que la comprensión no supere un umbral parametrizado se dará el ticket como no válido y se pedirá de nuevo. Se limitará a los usuarios el mal uso del módulo impidiendoles subir dos veces el mismo producto en el mismo día.
- Gamificación e incentivo de los usuarios para el crowdsourcing. Se premiará a los usuarios que hagan aportaciones dando puntos de reputación que desbloquean diferentes rangos dentro de la aplicación, insignias, funcionalidades avanzadas en filtros y análisis de compras y dando también más peso a sus aportaciones a la hora de establecer los precios. También se mostrará un ranking donde los usuarios podrán comparar su puntuación con la de otros usuarios.
- Algoritmo de establecimiento de precios en base a aportaciones de usuarios. Dando más peso a usuarios con mejor reputación y teniendo en cuenta el tiempo pasado entre aportaciones.
- Métricas básicas sobre variación de precios en productos y comercios, variación de precios en historial de compras del usuario. Existen métricas avanzadas disponibles para usuarios con alto rango de reputación.
- Sección de administración donde gestionar todas las entidades: usuarios, permisos, precios, comercios, aportaciones y poder auditar posibles malos usos del sistema.
- La categorización de productos debe ser automática y asegurar la calidad.

No funcionales

- Rendimiento y Escalabilidad: Tiempos de carga rápidos en dispositivos móviles y capacidad de respuesta ágil en consultas de bases de datos mediante Firebase Firestore.
- Escalabilidad de sistema: Aunque el objetivo inicial sea el despliegue en Sevilla el sistema debe ser genérico y estar preparado para una expansión controlada.
- Disponibilidad: Alta disponibilidad gracias a la infraestructura en la nube de Firebase Hosting y servicios backend asociados.
- Seguridad y Privacidad: Encriptación de credenciales de usuario, control de acceso basado en roles y almacenamiento seguro de imágenes de tickets.
- Usabilidad: Interfaz intuitiva y adaptada a patrones móviles modernos mediante Ionic, asegurando una experiencia fluida tanto en entornos Android como iOS.

- Mantener la calidad de los datos.
- Experiencia de usuario: La aplicación debe ser fácil de utilizar, con una navegación simple y orientada a la gamificación. Tomando como referencia aplicaciones como Duolingo se pretende hacer partícipe de las interfaces a una mascota, que hará de acompañante y guía en la aplicación.



Imagen generada mediante ChatGPT y no definitiva

Descripción de los módulos

Módulo de login: Gestiona el acceso de los usuarios a la plataforma, permitiendo la autenticación mediante correo electrónico y contraseña o mediante cuenta de Google de forma segura.
Módulo de gestión de cuenta: Administra el perfil del usuario, permitiendo visualizar su progreso de gamificación, sus puntos de reputación, rangos alcanzados, insignias obtenidas y un resumen de sus estadísticas personales de ahorro y aportaciones. También puede gestionar los datos de su cuenta, modificar su contraseña y darse de baja.
Ranking: Muestra una tabla de clasificación provincial (Sevilla en primera instancia) donde los usuarios pueden comparar su nivel de reputación y aportaciones con el de otros miembros de la comunidad, fomentando la participación activa.
Módulo de comercios: Administra la geolocalización y el alta de establecimientos cercanos. Permite marcar comercios favoritos, dar de alta nuevos comercios para su posterior validación y establecer áreas geográficas de interés para el usuario.

Módulo de gestión de listas: Facilita la creación, edición y guardado de listas de la compra. Se integra con el buscador para cruzar los ítems con los precios actuales y permite consultar historiales de listas pasadas o marcarlas como favoritas.
Módulo de búsqueda y comparación de productos: Permite buscar artículos por nombre, categoría, precio y cercanía. Muestra indicadores de fiabilidad basados en el porcentaje de seguridad (confirmaciones de la comunidad) y el tiempo transcurrido desde la última actualización.
Módulo de subida de productos/precios: Núcleo de la aplicación. Permite introducir precios manualmente o mediante la captura y procesamiento de tickets con LLM. Incluye un formulario de corrección para datos no comprendidos, filtros de control ante duplicidades diarias y rechazo automático si no se supera el umbral de confianza.
Módulo de métricas: Genera gráficos analíticos sobre la variación de precios en productos y comercios, así como la evolución del gasto en el historial de compras del usuario, desbloqueando vistas avanzadas según el rango de reputación.

Módulo de Administración y Auditoría

- Descripción y Operativa: Panel de control integral y exclusivo para roles con permisos elevados (ADMIN / MODERATOR), destinado a la supervisión, mantenimiento y seguridad de toda la plataforma.
- Flujo de interacción:
    - El usuario con rol administrador accede a la sección de gestión mediante credenciales verificadas por reglas de seguridad de Firestore.
    - Visualiza un cuadro de mando con KPIs globales de la aplicación (usuarios totales, tickets procesados hoy, incidencias de fraude, etc.).
    - Utiliza las herramientas de moderación para revisar comercios pendientes, auditar tickets sospechosos, gestionar usuarios y normalizar catálogos.
- Apartados:
    - Gestión de Usuarios y Roles: Asignación de roles (USER, MODERATOR, ADMIN), bloqueo de cuentas fraudulentas y revisión de historiales de puntos.
    - Creación de roles y gestión de permisos por rol.
    - Moderación de Contenido: Aprobación o rechazo de altas de nuevos comercios y revisión manual de tickets en cuarentena o reportados por la comunidad.
    - Configuración del Sistema: Ajuste de parámetros algorítmicos (pesos de reputación, umbrales de confianza del LLM y límites de aportaciones diarias por usuario).
    - Auditoría y Logs: Consulta de registros de actividad del sistema para detectar anomalías, fallos en Cloud Functions o intentos de manipulación de datos.

Política de Backups y Recuperación

Tanto durante la fase de desarrollo como en la versión definitiva en producción, se establece una estrategia de copias de seguridad automatizadas y restauración ante fallos sobre la infraestructura de Firebase para garantizar la integridad y continuidad de los datos.

- Protocolo y Frecuencia de Copias de Seguridad:
    - Firebase Firestore (Datos de aplicación): Configuración de exportaciones automáticas diarias mediante GCP Scheduled Queries / Cloud Scheduler hacia un bucket de Google Cloud Storage. Durante el desarrollo se programan backups diarios a última hora del día, mientras que en producción se mantiene la frecuencia diaria sumada a copias incrementales y retención de snapshots semanales y mensuales con política de almacenamiento multirregión.
    - Firebase Storage (Imágenes de tickets): Configuración de Bucket Lifecycle Rules con duplicación de almacenamiento en clase Nearline/Coldline para las imágenes procesadas y retención de originales durante el periodo de cuarentena/auditoría.
    - Código fuente y Configuración: Gestión de versiones integrada en repositorio centralizado (GitHub) con ramas protegidas (main y develop)

- Procedimiento y Pasos de Recuperación (Disaster Recovery):
    - Paso 1 (Detección e Aislamiento): Identificación del fallo o corrupción de datos mediante alertas automáticas de Firebase / Google Cloud Operations. En caso de brecha o corrupción masiva, activación del modo de mantenimiento desactivando las Cloud Functions de escritura.
    - Paso 2 (Selección del Punto de Restauración - RPO): Selección del snapshot válido más reciente (Point-in-Time Recovery) en Cloud Storage.
    - Paso 3 (Ejecución de la Restauración - RTO): Importación de los documentos de Firestore mediante la herramienta CLI de Google Cloud (gcloud firestore import) sobre una instancia de recuperación o sobre el proyecto principal tras purga de la colección afectada.
    - Paso 4 (Validación de Consistencia): Ejecución de scripts automáticos de verificación de integridad de referencias (usuarios-aportaciones-precios) antes de reabrir el tráfico.
    - Paso 5 (Reanudación del Servicio): Despliegue de reglas de seguridad actualizadas y reactivación de las puertas de enlace del backend.

Verificación y Validación

Con el fin de asegurar el cumplimiento de las especificaciones funcionales y no funcionales, se define un plan de pruebas estructurado enfocado en los siguientes escenarios críticos de comprobación:

- Validación de la Tasa de Confianza del Parser LLM:
    - Escenario: Procesamiento automatizado de tickets de compra mediante el modelo de lenguaje.
    - Comprobación: Envío de una batería de tickets de prueba (legibles, parcialmente borrosos, con iluminación deficiente) para verificar que el parser devuelve el porcentaje de fiabilidad parametrizado. Se validará que cuando el umbral de confianza cae por debajo del límite configurado, el sistema deriva automáticamente el ticket al formulario de corrección manual o rechaza el procesamiento según corresponda.

- Prueba del Algoritmo de Detección de Duplicados en el Mismo Día:
    - Escenario: Control de antifraude e inserción repetida de aportaciones.
    - Comprobación: Simulación de subida consecutiva del mismo ticket o producto por parte de un mismo usuario dentro de una misma ventana de 24 horas. Se verificará que el sistema intercepta el intento, bloquea la transacción, registra el intento de duplicidad y penaliza preventivamente la puntuación de reputación sin alterar el histórico de precios.
- Comprobación de Permisos de Acceso Restringidos al Rol ADMIN:
    - Escenario: Seguridad y control de acceso basado en roles (RBAC) en el panel de control.
    - Comprobación: Intento de acceso a los endpoints y vistas del Módulo de Administración y Auditoría desde cuentas con rol USER o peticiones no autenticadas. Se validará la denegación explícita de acceso (error 403 / Permission Denied), comprobando que la restricción se ejecuta de forma estricta tanto en el cliente (guards de Angular/Ionic) como en las reglas de seguridad del backend (firestore.rules).
- Consistencia de la Lectura de Precios NoSQL:
    - Escenario: Concurrencia de lecturas y actualizaciones en la base de datos de Firebase Firestore.
    - Comprobación: Verificación de la velocidad de respuesta y la consistencia eventual/inmediata del cálculo de la media ponderada de precios tras aportaciones simultáneas de múltiples usuarios. Se comprobará que las lecturas en la colección de productos reflejan los valores actualizados de manera coherente sin generar desincronizaciones en las vistas del comparador ni en los gráficos de métricas.

Procesos automatizados[1]

Denominación	Módulos que intervienen	Descripción del proceso
Gestión de precio de un producto	Módulo de subida de productos/precios, Módulo de búsqueda y comparación	Disparador: Subida de un ticket o aporte manual por parte del usuario.

[1] Se denomina así a procesos que el producto software ejecuta en forma independiente y que pueden reemplazar una operación que requiera intervención mínima del usuario. Generalmente se espera que impliquen la generación de registros nuevos en la BD del producto. Por ejemplo: generación automática de un pedido de materiales ante la detección de que el stock de un producto se encuentra por debajo de un límite X.
Opcionalmente se podrán incorporar en este apartado integraciones de APIs o servicios de terceros que brinden algún tipo de solución al problema del proyecto. Por ejemplo: integración con una plataforma de pagos online.

		Automatización: Ejecución del análisis mediante LLM, validación de umbrales de confianza, cálculo del peso según la reputación del usuario y actualización inmediata de la media ponderada del precio en Firebase Firestore si cumple las condiciones de seguridad.
Detección de actividad sospechosa de usuarios	Módulo de subida de productos/precios, Módulo de administración	Disparador: Intento de subida duplicada del mismo ticket/producto en el mismo día o desviaciones extremas de precios respecto a la media. Automatización: Bloqueo preventivo de la aportación, penalización temporal en los puntos de reputación del usuario y registro de una incidencia en el panel de administración para su auditoría.
Revisión y unificación de categorías y productos	Módulo de subida de productos/precios, Búsqueda y comparación, Base de datos global	Disparador: Ejecución periódica programada (semanal). Automatización: Análisis automatizado de la base de datos en busca de productos mal normalizados, duplicidades introducidas por variantes ortográficas de los usuarios o categorías redundantes, agrupándolas para mantener la calidad y consistencia del catálogo global.
Lectura de ticket con LLM	Módulo de subida de productos/precios	Disparador: Subida de ticket Automatización: Se enviará la imagen a un modelo LLM

		indicando que debe fijarse unicamente en productos, precios y el comercio. En base a eso y las categorías existentes, devolverá un formato JSON en el que indicará la lista de datos obtenidos y el porcentaje de fiabilidad.

Estimación de tamaño por módulo

Módulo	Porcentaje de participación / Producto
Módulo de Login y Gestión de Cuenta	10%
Módulo de Comercios y Geolocalización	10%
Módulo de Búsqueda y Comparación de Productos	15%
Módulo de Subida de Productos y Procesamiento LLM	20%
Módulo de Gestión de Listas y Optimización	15%
Módulo de Métricas y Gamificación (Ranking / Insignias)	10%
Módulo de Administración y Auditoría	20%
Total	100%

Módulos a programar (debe ser alrededor del 20% del proyecto):

Módulo
Módulo de Administración y Auditoría. Requiere control completo del modelo de datos y sistemas CRUD para cada entidad.

Entorno tecnológico y metodológico

Lenguajes de programación:	TypeScript, HTML5, SCSS
Framework:	Ionic (con Angular)
Arquitectura:	Arquitectura Cliente-Servidor desacoplada (Frontend híbrido con Ionic/Capacitor y Backend as a Service / Serverless con Firebase).
Motor de Base de Datos:	Base de datos NoSQL orientada a documentos (Firebase Firestore)
Metodología seleccionada:	Scrum (Jira)
Tipo de proyecto:	Sin cliente final

Planificación de actividades

Actividad	Fecha de inicio	Fecha de finalización
Análisis de requisitos, diseño funcional y arquitectura del sistema	29/10/2026	10/11/2026
Modelado de base de datos en Firebase y configuración inicial del entorno Ionic + Angular	11/11/2026	30/11/2026
Módulo CRUD de Usuarios y Roles (4 operaciones)	01/12/2026	18/12/2026
Programación del módulo de Auditoría y revisión de Tickets procesados por LLM (3 operaciones)	19/12/2026	08/01/2027
Programación de módulos CRUD para Gestión de Comercios (4 operaciones) y Normalización de Productos (4 operaciones)	09/01/2027	25/01/2027
Pruebas de integración de operaciones CRUD	26/01/2027	31/01/2027

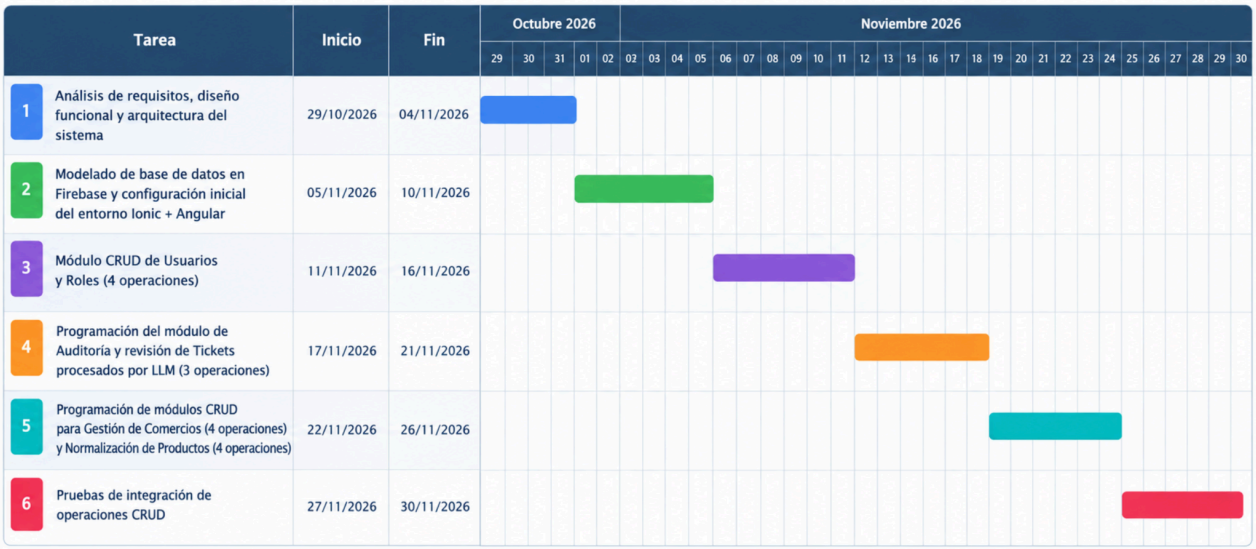


Imagen generada mediante ChatGPT